

AISM Policy-Format (Policy-as-Code für S2)

Formatspezifikation aism/v1alpha1 - GovernancePolicy

Dokumentstatus	Entwurf 0.1
Datum	05.10.2026
Bezug	../AISM-Spezifikation.md (insb. §6.2, §8, §9), policy.schema.json, policy.example.yaml
Normative Sprache	MUSS / DARF NICHT / SOLLTE / KANN gemäß RFC 2119

aism/v1alpha1 ist ein **AISM-eigener** Bezeichner und kein externer Standard. Die Schema-`$id` ist ein URN (`urn:aism:schema:policy:v1alpha1`) und verweist auf keine Domain. Alle Werte in `policy.example.yaml` (Rollen, Provider, Fristen, Schwellwerte, Modellnamen) sind **Beispiele**.

1. Zweck

Die Policy legt deklarativ fest, wie das Governance-Gateway (S2) an den Policy-Durchsetzungspunkten PEP-1 bis PEP-10 und beim Audit (AUD) entscheidet (AISM-Spezifikation §8). Sie wird

- als YAML-Datei in einem Git-Repository versioniert,
- vor dem Deployment gegen `policy.schema.json` (JSON Schema Draft 2020-12) validiert,
- vom Gateway beim Start und bei Änderung geladen (Mount `/etc/aism/policy.yaml`).

Die Policy enthält **keine Secrets**. Zugangsdaten werden nur über `secretRef` (`env:` oder `file:`) referenziert.

2. Aufbau im Überblick

```
apiVersion: aism/v1alpha1
kind: GovernancePolicy
metadata: { name, version, revision, owner, effectiveFrom, changeRef, signature }
spec:
  defaults:      # Bewertungsgrundsätze (default deny, fail-closed, local)
  subjects:      # Identitätsquellen, Rollen, Agenten
  piiDetectors:  # Regex/NER/Prüfsummen-Detektoren, Maskierungsstrategie
  dataClasses:   # Datenklassen mit Rang und Zuordnungsregeln
  routing:       # Provider-Allowlist, Routing-Regeln, Cloud-Fallback
  tools:         # Tool-Allowlist mit Argumentschema, Zugriffsart, Rate-Limits
  rag:          # Retrieval-Regeln, Collection-Zugriff pro Rolle
  webSearch:     # Freigabe der Web-Suche
  audit:         # Ereignisse, Aufbewahrung, Integrität, Entscheidungsprotokoll
```

3. Zuordnung zu den Kontrollpunkten der Spezifikation

Policy-Element	PEP / AUD (Spez. §8)	Kriterien (Spez. §9)
metadata.*	AUD (Policy-Version in jedem Audit-Eintrag)	S-10, S-11
spec.defaults	PEP-3	M-12
spec.subjects	PEP-1	S-01
spec.piiDetectors	PEP-2, PEP-8, PEP-10	M-02
spec.dataClasses	Eingang für PEP-3 bis PEP-7	-
spec.routing	PEP-4, PEP-9	M-04
spec.tools	PEP-7, PEP-8	M-07, M-08, S-03, S-09
spec.rag	PEP-5	M-09, M-10
spec.webSearch	PEP-6	M-11
spec.audit	AUD	M-05, S-02

4. Elemente im Detail

4.1 Versionierung und Metadaten (`apiVersion`, `kind`, `metadata`)

Feld	Pflicht	Bedeutung
<code>apiVersion</code>	ja	Formatversion, aktuell <code>aism/v1alpha1</code> . Inkompatible Änderungen erhöhen die Formatversion.
<code>kind</code>	ja	<code>GovernancePolicy</code>
<code>metadata.name</code>	ja	Name der Policy
<code>metadata.version</code>	ja	Inhaltsversion nach Semantic Versioning 2.0.0. Major : restriktivere oder inkompatible Semantik; Minor : neue Regeln/Tools; Patch : Korrekturen ohne Wirkungsänderung.
<code>metadata.revision</code>	nein (signiert: ja)	Git-Commit-SHA, von der CI gesetzt, nicht manuell. Ohne Git setzt <code>tools/aism-policy-sign.py</code> eine Inhalts-Revision (erste 40 Hex-Zeichen des SHA-256 der Datei ohne Revisionszeile). Signierte Policies ohne <code>revision</code> lehnt das Gateway ab.
<code>metadata.owner</code>	ja	Verantwortliche Stelle
<code>metadata.effectiveFrom</code>	nein	Frühester Gültigkeitszeitpunkt (RFC 3339)
<code>metadata.changeRef</code>	nein	Verweis auf den Merge-/Pull-Request
<code>metadata.signature</code>	nein (K3: ja)	<code>method</code> , <code>ref</code> , <code>signer</code> . ssh-sig (vom Referenz-Gateway geprüft, §6.1): abgesetzte SSHSIG-Signatur (Ed25519, Namespace <code>aism-policy</code>) über die Policy-Datei; <code>ref</code> = Dateiname der Signatur im selben Verzeichnis (z. B. <code>policy.yaml.sig</code>), <code>signer</code> = Prinzipal aus <code>allowed_signers</code> (Pflicht). <code>git-tag-gpg</code> , <code>git-tag-ssh</code> , <code>sigstore-cosign</code> sind nur deklarativ; das Referenz-Gateway verifiziert sie nicht und lehnt sie bei Signaturpflicht ab.

Das Gateway MUSS `name`, `version`, `revision` und den SHA-256-Digest der geladenen Datei in jedem Audit-Eintrag mitschreiben (Ereignis `policy.loaded` beim Laden).

4.2 Bewertungsgrundsätze (`spec.defaults`)

Feld	Wert	Hinweis
<code>decision</code>	<code>deny</code> (fest)	Default deny; nicht konfigurierbar

Feld	Wert	Hinweis
route	local (fest)	Lokales Routing ist Standard
failMode	closed (fest)	Bei Fehlern in Erkennung, Policy-Auswertung oder Laden: Ablehnung
maxToolRounds	Ganzzahl	Obergrenze der Tool-Runden je Request (S-09)
maxRequestBytes	Ganzzahl	Größenlimit (HTTP 413 bei Überschreitung)

Die festen Werte sind im Schema als `const` definiert. Eine Policy, die sie ändert, ist **ungültig** und wird nicht geladen.

4.3 Identitäten, Rollen, Agenten (`spec.subjects`) - PEP-1

- `identitySources` beschreibt, wie das Gateway Identitäten prüft:
- `oidc-bearer`: JWT im Authorization-Header, Prüfung über `issuer`, `audience` und `jwksUri`.
- `forwarded-jwt-hs256`: signiertes JWT in einem Header (z. B. von Open WebUI ab v0.9.6 über `ENABLE_FORWARD_USER_INFO_HEADERS` mit `FORWARD_USER_INFO_HEADER_JWT_SECRET`), geprüft mit dem gemeinsamen Secret aus `secretRef` und optional `issuer` (Open WebUI: `open-webui`). `client` bindet die Quelle an einen per `api-key` authentifizierten Client: Das JWT wird nur von diesem Client akzeptiert.
- `api-key`: Client- bzw. Dienst-zu-Dienst-Schlüssel als Bearer-Token (z. B. für Open WebUI, n8n oder Perplexica); `client` benennt den Client.
- `roles` bildet Claims auf Rollen ab (`match.claim`, `match.anyOf`). Ein Subjekt kann mehrere Rollen haben. Das JWT von Open WebUI enthält `sub`, `email`, `name`, `role`, `iss`, `iat`, `exp`, aber **keine Gruppen**; für Open-WebUI-Benutzer ist daher nur der Claim `role` (`user`, `admin`) nutzbar. Gruppenbasierte Rollen setzen eine Quelle mit Gruppen-Claim voraus (z. B. `oidc-bearer`).
- Ohne zugeordnete Rolle greift `defaults.decision: deny` (keine passende Routing-Regel).
- `agents` listet zulässige Agenten-IDs. Requests mit unbekannter Agent-ID erhalten keine Tool-Rechte.

4.4 PII-Detektoren und Maskierung (`spec.piiDetectors`) - PEP-2, PEP-8, PEP-10

Feld	Bedeutung
entity	Entitätstyp, z. B. <code>EMAIL</code> , <code>PERSON</code> , <code>SECRET</code> , <code>IBAN</code>
type	<code>regex</code> (Muster), <code>ner</code> (Named Entity Recognition per Modell), <code>checksum</code> (Muster plus Prüfsumme, z. B. IBAN Mod-97, Luhn)
patterns	Reguläre Ausdrücke (für <code>regex</code> und <code>checksum</code>). Enthält ein Muster die benannte Gruppe (<code>?P<pii>...</code>), wird nur diese Gruppe maskiert (Beispiel <code>person-title</code> : „Frau Dr. Müller“ → „Frau Dr.“). Bei <code>checksum</code> gilt die längste prüfsummengültige Teilspanne, die an einer Token-Grenze endet (sonst würden z. B. auf eine IBAN folgende Großbuchstaben-Token die Prüfung scheitern lassen).
ner	Modell, Labels, Mindest-Score, Sprachen. Der Gateway-Prototyp unterstützt <code>model: "spacy:<modellname>"</code> (Beispiel: <code>spacy:xx_ent_wiki_sm</code>); <code>minScore</code> wird dort nicht ausgewertet, weil spaCy-Kleinmodelle keinen Score je Entität liefern. Ist das Modell nicht ladbar, verweigert das Gateway alle Requests (fail-closed).
applyTo	Anwendungsorte: <code>prompt</code> , <code>tool_result</code> , <code>rag_ingest</code> , <code>web_query</code> , <code>response</code>
masking.strategy	<code>placeholder</code> (nummerierter Platzhalter), <code>redact</code> (entfernen), <code>hash</code> (Hash-Wert)
masking.placeholderFormat	z. B. <code><EMAIL_{n}></code> ; <code>{n}</code> wird pro Request fortlaufend vergeben
masking.reversible	<code>true</code> : Zuordnung Platzhalter ↔ Klartext wird request-gebunden im Speicher gehalten. Nur bei <code>placeholder</code> zulässig.
masking.demaskFor	Rollen, für die die Antwort im Rückweg demaskiert wird (PEP-10)

Regeln:

- Platzhalter-Zuordnungen MÜSSEN request-gebunden, nur im Speicher und mit begrenzter Lebensdauer gehalten werden (Spez. §6.2).
- Für SECRET-Entitäten SOLLTE `reversible: false` gelten; Secrets werden nie demaskiert.
- Gleicher Klartext im selben Request erhält denselben Platzhalter.
- Überschneiden sich Treffer mehrerer Detektoren, gilt der längste Treffer; bei gleicher Länge die Entität der höheren Datenklasse.
- Erkennungsqualität (insbesondere NER) ist **nicht garantiert** und MUSS vom Betreiber mit eigenen Testdaten gemessen werden (Spez. §10). Referenzmessung auf synthetischen deutschen Daten: [../conformance/pii-eval/README.md](https://conformance/pii-eval/README.md) (Namen: Recall maskiert 0,86 auf dem Held-out-Set).

4.5 Datenklassen (`spec.dataClasses`)

Jede Datenklasse hat einen `rank` (höher bedeutet schutzbedürftiger). Für einen Request wird die **höchste zutreffende Klasse** bestimmt aus

- `detectedEntities`: im Request erkannte Entitäten (vor der Maskierung festgestellt),
- `collections`: angefragte bzw. abgerufene RAG-Collections,
- `roles`: Rollen des Subjekts,
- `always`: Basisklasse.

Die Datenklasse kann sich im Verlauf eines Requests **erhöhen** (z. B. durch RAG-Treffer aus einer `confidential`-Collection oder Tool-Ergebnisse), aber nie verringern. Routing-Entscheidungen werden bei einer Erhöhung neu ausgewertet.

4.6 Routing und Cloud-Fallback (`spec.routing`) - PEP-4, PEP-9

Feld	Bedeutung
<code>cloudEnabled</code>	Globaler Schalter. <code>false</code> bedeutet: kein Cloud-Egress, unabhängig von Regeln.
<code>providers[]</code>	Provider-Allowlist : nur hier gelistete Endpunkte sind Routing-Ziele. Cloud-Provider MÜSSEN <code>https://</code> und ein <code>credentialRef</code> haben.
<code>rules[].priority</code>	0-1000; höhere Priorität wird zuerst ausgewertet.
<code>rules[].when</code>	Bedingungen (<code>dataClasses</code> , <code>roles</code> , <code>agents</code> , <code>models</code>); alle angegebenen Bedingungen müssen zutreffen (UND); Listen innerhalb einer Bedingung sind ODER.
<code>rules[].route.mode</code>	<code>local-only</code> , <code>local-with-cloud-fallback</code> oder <code>deny</code>
<code>rules[].route.cloudProviders</code>	Allowlist für den Fallback dieser Regel (Teilmenge von <code>providers</code>)
<code>rules[].route.fallbackOn</code>	Auslöser: <code>local_unavailable</code> , <code>local_overloaded</code> , <code>context_too_large</code> , <code>model_not_available_locally</code>

Ein Cloud-Fallback erfolgt nur, wenn **alle** folgenden Bedingungen erfüllt sind: `cloudEnabled: true`, die gewählte Regel hat `local-with-cloud-fallback`, ein Auslöser aus `fallbackOn` liegt vor, der Provider steht in `cloudProviders`, und die Maskierung war erfolgreich (fail-closed). Das Ergebnis wird als `egress.cloud` auditiert (mit Regel-ID, Provider-ID und Payload-Hash).

4.7 Tools (`spec.tools`) - PEP-7, PEP-8

Feld	Bedeutung
<code>id</code>	Funktionsname, identisch mit <code>tools[].function.name</code> im Chat-Completions-Request
<code>gateway / target</code>	<code>n8n-webhook</code> + Webhook-Pfad oder <code>mcp</code> + MCP-Server-ID
<code>access</code>	<code>read</code> oder <code>write</code>
<code>parameterized</code>	immer <code>true</code> : Argumente werden gebunden (z. B. SQL-Parameter <code>\$1</code>), nie in Code oder Abfragetext interpoliert

Feld	Bedeutung
<code>argumentsSchema</code>	JSON Schema der Argumente; MUSS <code>type: object</code> und <code>additionalProperties: false</code> haben
<code>allow.roles / allow.agents</code>	Allowlist pro Rolle und Agent; beide müssen zutreffen, falls angegeben
<code>allow.maxDataClassRank</code>	Tool nur nutzbar, wenn die Datenklasse des Requests diesen Rang nicht übersteigt
<code>rateLimit</code>	<code>requests pro per</code> (z. B. <code>1m</code>) je <code>scope</code> (<code>subject</code> , <code>agent</code> , <code>global</code>); Überschreitung führt zu einer <code>role: "tool"</code> -Fehlermeldung und <code>tool.call.denied</code>
<code>requireConfirmation</code>	Benutzerbestätigung vor Ausführung; für <code>write</code> Pflichtfeld. Ablauf im Referenz-Stack: S3 führt das Tool nicht aus, sondern legt eine ausstehende Aktion an (<code>tool.call.pending</code> , Gültigkeit <code>CONFIRMATION_TTL_SECONDS</code> , Standard 900 s) und meldet S6 <code>{"status": "pending_confirmation", "confirmation_id": ...}</code> . Nur dasselbe Subjekt kann sie über S2 bestätigen oder ablehnen (GET <code>/aism/v1/confirmations</code> , POST <code>/aism/v1/confirmations/{id}/approve</code> bzw. <code>/reject</code>). Bei Bestätigung prüft S3 Tool-Freigabe (aktuelle Policy, Rollen), Schema und Rate-Limit erneut, führt einmalig aus (<code>tool.call.confirmed</code> , <code>tool.result</code>) und gibt das maskierte Ergebnis zurück; Wiederholung → 409. Argumente werden so ausgeführt, wie das Modell sie erzeugt hat – also ggf. mit Platzhaltern statt Klartext.
<code>maskResult</code>	immer <code>true</code> : Ergebnis läuft vor Rückgabe an S6 durch die PII-Maskierung

Nur Tools, die für das Subjekt zulässig sind, werden S6 überhaupt im `tools`-Array angeboten. Unabhängig davon prüft PEP-7 jeden zurückkommenden `tool_calls`-Eintrag erneut (Name, Schema, Rate-Limit, Datenklasse).

4.8 RAG (`spec.rag`) – PEP-5

Feld	Bedeutung
<code>requireMaskedQuery</code>	immer <code>true</code> : Embedding-Anfragen und Vektorsuchen erfolgen ausschließlich mit maskiertem Text (nach PEP-2)
<code>maskOnIngest</code>	Maskierung bei der Ingestion vor dem Embedding
<code>embeddingModel</code>	Name des Embedding-Modells; muss für Ingestion und Query identisch sein (M-09)
<code>maxChunks</code>	Obergrenze Top-k
<code>collections[].allow.roles</code>	Collection-Zugriff pro Rolle
<code>collections[].filter</code>	Pflichtfilter für die Qdrant-Query: <code>payloadKey</code> (z. B. <code>access_group</code>) und Werte aus Rollen oder Claim (M-10)
<code>collections[].dataClass</code>	Datenklasse, die Treffer aus dieser Collection dem Request zuweisen

4.9 Web-Suche (`spec.webSearch`) – PEP-6

`enabled: false` deaktiviert S5 vollständig (M-11). `allow` beschränkt nach Rolle und maximalem Datenklassen-Rang (Datenklasse des eingehenden Requests). `stripPlaceholders` entfernt Platzhalter vor der Suchanfrage, `maxResults` begrenzt die Treffer.

Referenz-Stack: Ist die Suche für den Request zulässig, bietet S3 dem Modell das eingebaute Tool `web_search` (`{"query": string}`) an; der Name ist reserviert und darf nicht als `tools.definitions[].id` verwendet werden. Jede vom Modell erzeugte Suchanfrage wird erneut über S2 maskiert (Kontext `web_query`), Platzhalter werden entfernt, dann geht sie an SearXNG (`format=json`). Ergebnisse werden maskiert und als `{"untrusted_web_results": ...}` gekennzeichnet zurückgegeben (S-05); Audit `egress.websearch` mit SHA-256 der Anfrage. Ist die Suche nicht zulässig, wird `web_search` nicht angeboten und ein dennoch erzeugter Aufruf mit `tool.call.denied` (`web_search_not_permitted`) abgelehnt.

4.10 Audit und Logging (spec.audit) - AUD

Feld	Bedeutung
sink	file-jsonl (Pfad), syslog oder otlp-logs (Endpunkt)
events	zu protokollierende Ereignistypen (siehe Schema); neu in dieser Fassung: tool.call.pending, tool.call.confirmed, tool.call.rejected (Bestätigungsfluss)
plaintextPII	immer false: Audit-Einträge enthalten Entitätstypen und Zähler, keine Klartextwerte (M-05)
storePayloadHash	SHA-256 des maskierten Payloads, z. B. als Nachweis bei Cloud-Egress
retention.days	Aufbewahrungsdauer; basis verweist auf die interne Vorgabe. Rechtliche Fristen legt dieses Format nicht fest.
integrity	Hash-Verkettung der Einträge (S-02)
decisionLog	Entscheidungsprotokoll mit Regel-Trace (siehe §5.3)
traceContext	w3c-traceparent: Trace-ID aus dem W3C-Trace-Context-Header traceparent wird in jeden Eintrag übernommen (S-10)

Pflichtfelder eines Audit-Eintrags: ts, request_id, trace_id (falls vorhanden), subject, event, policy (name, version, revision, digest), decision, rule_ids.

5. Bewertungssemantik

5.1 Grundregeln

1. **Default deny:** Was nicht ausdrücklich erlaubt ist, ist verboten (Modelle, Provider, Tools, Collections, Web-Suche).
2. **Fail-closed:** Ist die Policy ungültig, nicht ladbar oder schlägt ein PEP technisch fehl, lehnt das Gateway ab (Egress immer; lokale Verarbeitung nur, wenn keine Maskierung oder Policy-Prüfung ausgefallen ist).
3. **Monotonie der Datenklasse:** Die Datenklasse eines Requests kann während des Requests nur steigen.

5.2 Vorrang

Bereich	Regel
Routing	Regeln werden nach priority absteigend geprüft; die erste zutreffende Regel gilt (first match). Gleiche Priorität bei überlappenden Bedingungen ist ein Validierungsfehler. Trifft keine Regel zu, gilt deny.
deny vs. allow	Eine zutreffende deny-Regel kann nicht durch eine Regel mit niedrigerer Priorität aufgehoben werden.
Globale Schalter	routing.cloudEnabled: false und webSearch.enabled: false haben Vorrang vor allen Regeln.
Tools, Collections	Erlaubnis nur bei Treffer in der Allowlist und Einhaltung aller Bedingungen (Rolle, Agent, Datenklasse, Rate-Limit).

5.3 Entscheidungsprotokoll (Explain)

Für jede Entscheidung erzeugt das Gateway einen Eintrag mit geprüften Regeln und Ergebnis, ohne Klartext-PII:

```
{
  "ts": "2026-10-05T10:15:02+02:00",
  "request_id": "req_7f3c",
  "trace_id": "4bf92f3577b34da6a3ce929d0e0e4736",
  "event": "route.decided",
  "subject": "user:4711",
  "roles": ["it-ops", "staff"],
  "data_class": "confidential",
  "detected_entities": {"EMAIL": 1},
  "policy": {"name": "aism-default", "version": "1.0.0", "revision": "3f2a9c1", "digest":
"sha256:..."},
  "trace": [
    {"rule": "restricted-local-only", "priority": 900, "matched": false, "reason": "data_class !=
restricted"},
    {"rule": "confidential-local-only", "priority": 800, "matched": true}
  ],
  "decision": "local-only",
  "provider": "local-llama"
}
```

Das Gateway SOLLTE zusätzlich einen Dry-Run-Modus anbieten (AISM-intern, z. B. `aism policy explain --input request.json`), der dieselbe Ausgabe ohne Ausführung liefert.

6. Änderungsprozess (Git-Review)

1. **Repository:** Policy liegt in einem eigenen Repository oder Verzeichnis mit `CODEOWNERS`. Owner sind die in `metadata.owner` genannte Stelle und der Betrieb.
2. **Änderung** nur per Merge-/Pull-Request; direkte Pushes auf den Hauptbranch sind gesperrt (Branch-Protection).
3. **CI-Prüfungen** (MÜSSEN bestehen): - Schema-Validierung gegen `policy.schema.json`, - semantische Prüfungen (eindeutige IDs, Referenzen auf existierende Rollen/Provider/Collections, keine überlappenden Regeln gleicher Priorität, kompilierbare Regex), - Policy-Tests (Beispiel-Requests mit erwarteter Entscheidung), - relevante Tests der Konformitätssuite (`./conformance/AISM-Konformitaet.md`).
4. **Review:** mindestens eine Freigabe durch einen Code-Owner; Änderungen an `routing`, `tools` mit `access: write` oder `audit` SOLLTEN zwei Freigaben erfordern.
5. **Versionierung:** `metadata.version` wird gemäß §4.1 erhöht; CI setzt `metadata.revision`.
6. **Signatur** (K3): nach dem Merge signiert eine berechtigte Stelle die Datei (§6.1); das Gateway lädt bei Signaturpflicht nur Policies mit gültiger Signatur.
7. **Rollout:** Das Gateway lädt die neue Datei, prüft zuerst die Signatur, dann Schema und Semantik, und protokolliert `policy.loaded` mit Version, Revision, Digest und Signaturinformation. Bei Fehlern (auch: unsigniert, Signatur ungültig, unbekannter Schlüssel, Rollback) bleibt die vorherige gültige Version aktiv, und `policy.load_failed` wird protokolliert (`kept_active: true`). Ohne gültige Policy beim Start: fail-closed (alle Requests 503).
8. **Rollback:** Revert-Commit plus neuer Patch-Release mit **neuem** `effectiveFrom` ($\geq$ dem der aktiven Policy); kein manuelles Editieren im laufenden Container. Das Gateway lehnt signierte Policies ab, deren `effectiveFrom` vor dem der aktiven Policy liegt (Schutz gegen das Wiedereinspielen alter, gültig signierter Versionen).

6.1 Signatur und Verifikation (`ssh-sig`)

Gewählt wurde das **OpenSSH-Signaturformat SSHSIG** (`ssh-keygen -Y sign`, `PROTOCOL.sshsig`) mit **Ed25519**: vollständig offline (keine Transparenz-Logs, kein OIDC-Zertifikatsdienst wie bei Sigstore keyless), verbreitete Werkzeuge, Schlüssel auch für Git-Signaturen (`gpg.format=ssh`) nutzbar, Prüfung im Gateway ohne externe Binärdateien (`aism_gateway/policysig.py`, nur `cryptography`). Sigstore/cosign bleibt als deklarative Methode möglich, wird aber vom Referenz-Gateway nicht geprüft.

Element	Festlegung
Signierte Daten	exakte Bytes der Policy-Datei (inkl. <code>metadata.revision</code> und <code>metadata.signature</code> ; die Signatur liegt abgesetzt in <code>ref</code> , daher kein Zirkelschluss)
Namespace	<code>aism-policy</code> (verhindert, dass z. B. eine Git- oder Datei-Signatur desselben Schlüssels als Policy-Signatur gilt)
Hash	<code>sha512</code> (Standard von <code>ssh-keygen</code>), <code>sha256</code> wird akzeptiert
Schlüssel	nur <code>ssh-ed25519</code> (keine RSA-/ECDSA-/sk- -Schlüssel, keine Zertifikate)
Vertrauensanker	OpenSSH- <code>allowed_signers</code> -Datei, nicht Teil der Policy, sondern Gateway-Konfiguration (<code>POLICY_ALLOWED_SIGNERS</code> , getrennt read-only gemountet). Zeilen mit <code>namespaces="..."</code> gelten nur, wenn <code>aism-policy</code> enthalten ist; <code>cert-authority</code> , <code>valid-after/-before</code> werden nicht ausgewertet (Zeile wird ignoriert).
Signaturpflicht	<code>POLICY_REQUIRE_SIGNATURE=true</code> (Compose-Standard). Ohne Pflicht prüft das Gateway eine deklarierte Signatur trotzdem, wenn ein Vertrauensanker konfiguriert ist.
Reihenfolge	Signatur → Schema → Semantik → Rollback-Prüfung; erst dann Aktivierung
Status	<code>GET /aism/v1/policy</code> liefert <code>revision</code> , <code>signature.verified</code> , <code>signature.required</code> , <code>signer</code> , Schlüssel-Fingerprint und ggf. <code>last_load_error</code>

Werkzeug: `../tools/aism-policy-sign.py`

```
# einmalig, AUSSERHALB des Repositorys (privaten Schlüssel nie committen; besser: Hardware-
Token/HSM-gestützter Signierplatz)
python3 tools/aism-policy-sign.py keygen --out ~/.aism-policy-keys --identity policy-
signer@example.com
cp ~/.aism-policy-keys/allowed_signers config/policy-trust/allowed_signers

# je Release: setzt metadata.revision (Git-HEAD oder Inhalts-Revision) und metadata.signature,
schreibt policy.yaml.sig
python3 tools/aism-policy-sign.py sign --key ~/.aism-policy-keys/policy-signing.key \
    --identity policy-signer@example.com policy/policy.yaml
python3 tools/aism-policy-sign.py verify --allowed-signers config/policy-trust/allowed_signers
policy/policy.yaml
# gleichwertig mit OpenSSH:
ssh-keygen -Y verify -f config/policy-trust/allowed_signers -I policy-signer@example.com \
    -n aism-policy -s policy/policy.yaml.sig < policy/policy.yaml
```

Mit `ssh-keygen -Y sign -n aism-policy -f <key> policy.yaml` erzeugte Signaturen werden ebenfalls akzeptiert (die Metadaten `revision/signature` müssen dann vorher von Hand gesetzt werden). Der Signierer schreibt zuerst `policy.yaml.sig`, dann `policy.yaml` (jeweils atomar); das Gateway beobachtet beide Dateien und lädt nach jeder Änderung neu – ein Zwischenzustand wird abgelehnt, die alte Version bleibt aktiv. Policy und Vertrauensanker werden als **Verzeichnis** gemountet, damit atomare Ersetzungen sichtbar werden (Einzeldatei-Bind-Mounts sehen sie nicht).

Grenzen: keine Schlüsselrotation/-sperrung außer durch Ändern von `allowed_signers`; keine Mehrfachsignatur (Vier-Augen-Prinzip erfolgt im Git-Review, nicht kryptografisch); der Signierer bestätigt nur „diese Bytes“, nicht die inhaltliche Prüfung.

7. Optionale Abbildung auf OPA/Rego

Die Policy kann statt durch die eingebaute Engine des Gateways durch **Open Policy Agent (OPA)** ausgewertet werden. Die YAML-Policy wird dabei als **Daten** (`data.aism`) geladen; die Logik liegt in Rego. Die folgenden Beispiele verwenden die Rego-Syntax von OPA 1.x und sind **Skizzen, keine vollständige Implementierung**.

Tool-Allowlist (PEP-7)

```

package aism.tools

default allow := false

tool := t if {
  some t in data.aism.spec.tools.definitions
  t.id == input.tool_call.name
}

allow if {
  some role in input.subject.roles
  role in tool.allow.roles
  input.agent in tool.allow.agents
  input.data_class_rank <= tool.allow.maxDataClassRank
  input.arguments_valid          # JSON-Schema-Prüfung erfolgt vorab im Gateway
  not input.rate_limited
}

deny_reason contains "tool_not_permitted" if not tool
deny_reason contains "rate_limited" if input.rate_limited

```

Routing (PEP-4), first match nach Priorität

```

package aism.routing

# Regeln werden vom Gateway vor dem Laden nach priority absteigend sortiert
# und als data.aism.routing_rules_sorted bereitgestellt.
rules := data.aism.routing_rules_sorted

matches(r) if {
  every k, v in r.when {
    cond(k, v)
  }
}

cond("dataClasses", v) if input.data_class in v
cond("roles", v) if { some r in input.subject.roles; r in v }
cond("agents", v) if input.agent in v
cond("models", v) if input.model in v

decision := r.route if {
  some i
  r := rules[i]
  matches(r)
  not earlier_match(i)
}

earlier_match(i) if { some j; j < i; matches(rules[j]) }

cloud_allowed if {
  data.aism.spec.routing.cloudEnabled
  decision.mode == "local-with-cloud-fallback"
  input.fallback_trigger in decision.fallbackOn
}

```

Die Sortierung nach `priority` erfolgt bei der Datenaufbereitung vor dem Laden in OPA. Abbildung der übrigen Elemente:

Policy-Element	Rego-Paket (Vorschlag)	Eingabe (<code>input</code>)
<code>subjects</code>	<code>aism.authz</code>	validierte Claims

Policy-Element	Rego-Paket (Vorschlag)	Eingabe (<code>input</code>)
<code>routing</code>	<code>aism.routing</code>	Datenklasse, Rollen, Agent, Modell, Fallback-Auslöser
<code>tools</code>	<code>aism.tools</code>	Tool-Call, Validierungsergebnis, Rate-Limit-Status
<code>rag</code>	<code>aism.rag</code>	Collection, Rollen → erlaubte Filterwerte
<code>webSearch</code>	<code>aism.websearch</code>	Rollen, Datenklasse

PII-Erkennung, Maskierung und Audit-Schreiben bleiben im Gateway, da sie Daten transformieren bzw. persistieren; OPA liefert nur Entscheidungen. OPAs eigene Decision Logs KÖNNEN zusätzlich genutzt werden, ersetzen aber nicht das AISM-Audit (`plaintextPII: false` gilt auch dort; OPA-Input-Masking ist entsprechend zu konfigurieren).

8. Validierung

```
# Beispiel mit check-jsonschema (pip install check-jsonschema)
check-jsonschema --schemafile policy/policy.schema.json policy/policy.example.yaml
```

Das JSON Schema prüft Struktur und feste Werte. Semantische Prüfungen (§6 Punkt 3) sind zusätzlich erforderlich und nicht im Schema ausdrückbar. Der Gateway-Prototyp (`../gateway/aism_gateway/policy.py`) setzt sie um; abweichend von §6 lehnt er **jede** doppelte Priorität ab, nicht nur bei überlappenden Bedingungen (strenger, einfacher prüfbar).